

Task Descriptor-1: (Setup Fast API Backend)

Instructions:

- Install FastAPI and Uvicorn.
- Use AI to generate a basic FastAPI application with one endpoint

Sample Code:

```
from fastapi import FastAPI
app = FastAPI()
@app.get("/")
def read_root():
    return {"message": "Welcome to AI-powered FastAPI"}
```

Expected Output:

Running the server and accessing:

`http://127.0.0.1:8000/`

Response:

```
{"message": "Welcome to AI-powered FastAPI"}
```

Prompt:-

Generate a simple FastAPI application in Python.

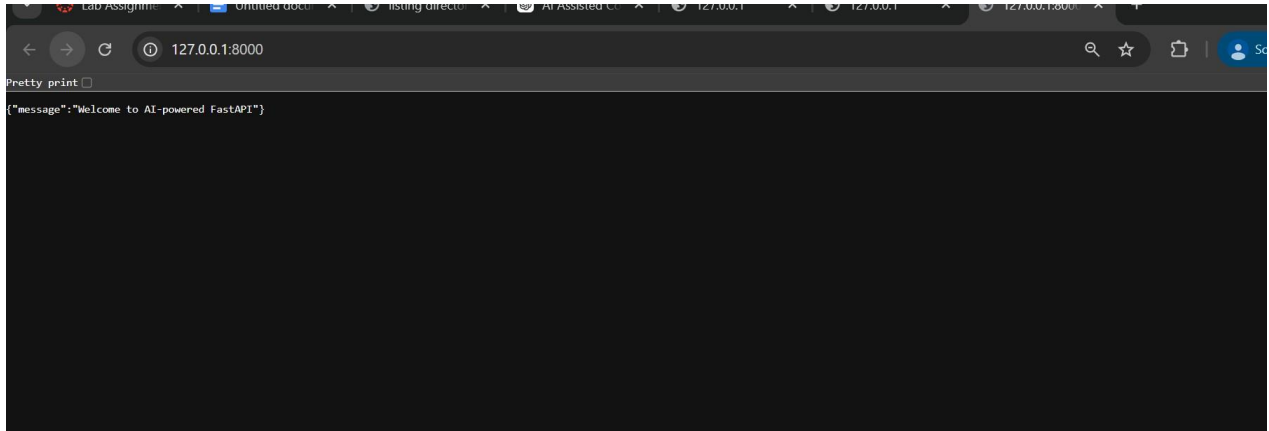
Requirements:

- Import FastAPI
- Create FastAPI app
- Create one GET endpoint "/"
- The endpoint should return JSON message:

```
{"message": "Welcome to AI-powered FastAPI"}
```

The code should run using uvicorn and be beginner friendly.

Output:-



Task Descriptor- 2: (API – Create and Read Items)

Instructions:

- Implement endpoints to add and retrieve products.
- Use a Python list as temporary storage.

Sample Code:

```
items = []  
@app.get("/items")  
def get_items():  
    return items  
@app.post("/items")  
def add_item(item: dict):  
    items.append(item)  
    return {"message": "Item created", "item": item}
```

Expected Output:

GET /items → []

POST /items with {"name":"Pen","price":20}

→ {"message":"Item created","item":{"name":"Pen","price":20}}

```
localhost:3000/items
Pretty print ☒
[{"name": "Pen", "price": 20}]
```

Task Descriptor- 3: (Update Existing Item)

Instructions:

- Use AI to generate an endpoint to update an item using its index.

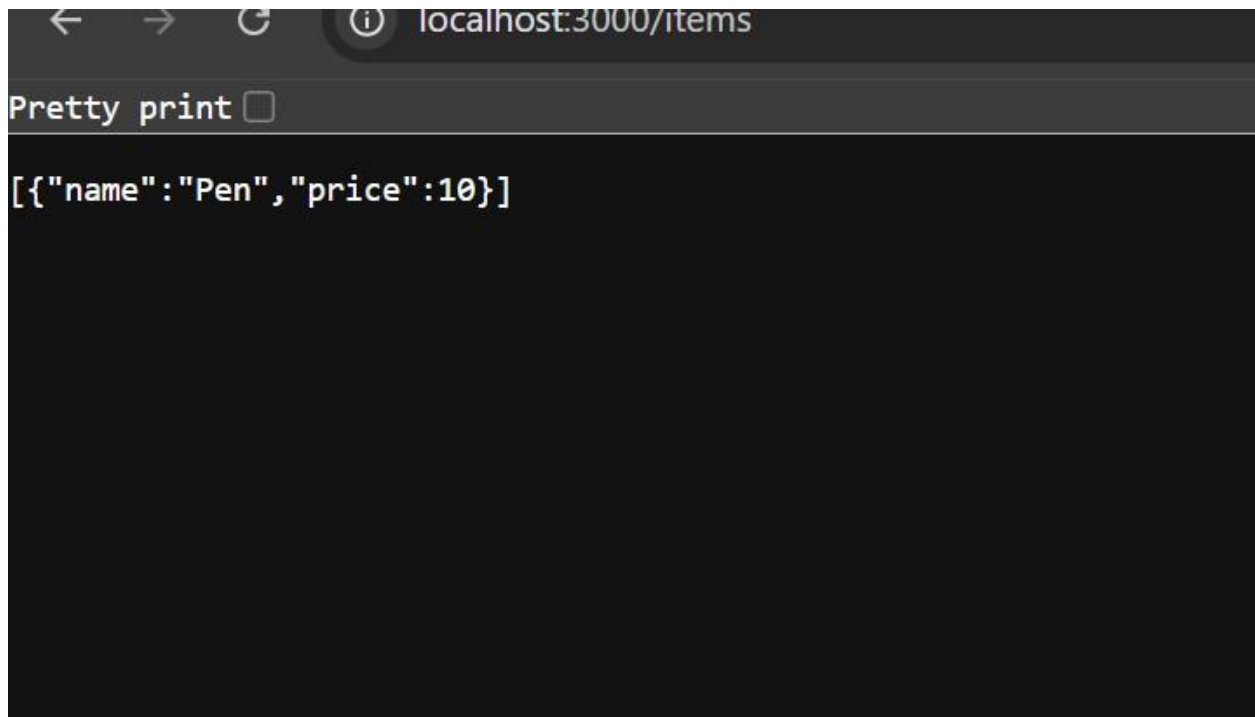
Sample Code:

```
@app.put("/items/{index}")
def update_item(index: int, item: dict):
    if index >= len(items):
        return {"error": "Item not found"}
    items[index] = item
    return {"message": "Item updated", "item": item}
```

Expected Output:

PUT /items/0 with {"name": "Pencil", "price": 10}

→ {"message": "Item updated", "item": {"name": "Pencil", "price": 10}}



Task Descriptor- 4: (Delete Item)

Instructions:

- Implement deletion of an item by index.

Sample Code: `@app.delete("/items/{index}")`

```
def delete_item(index: int):
```

```
    if index >= len(items):
```

```
        return {"error": "Item not found"}
```

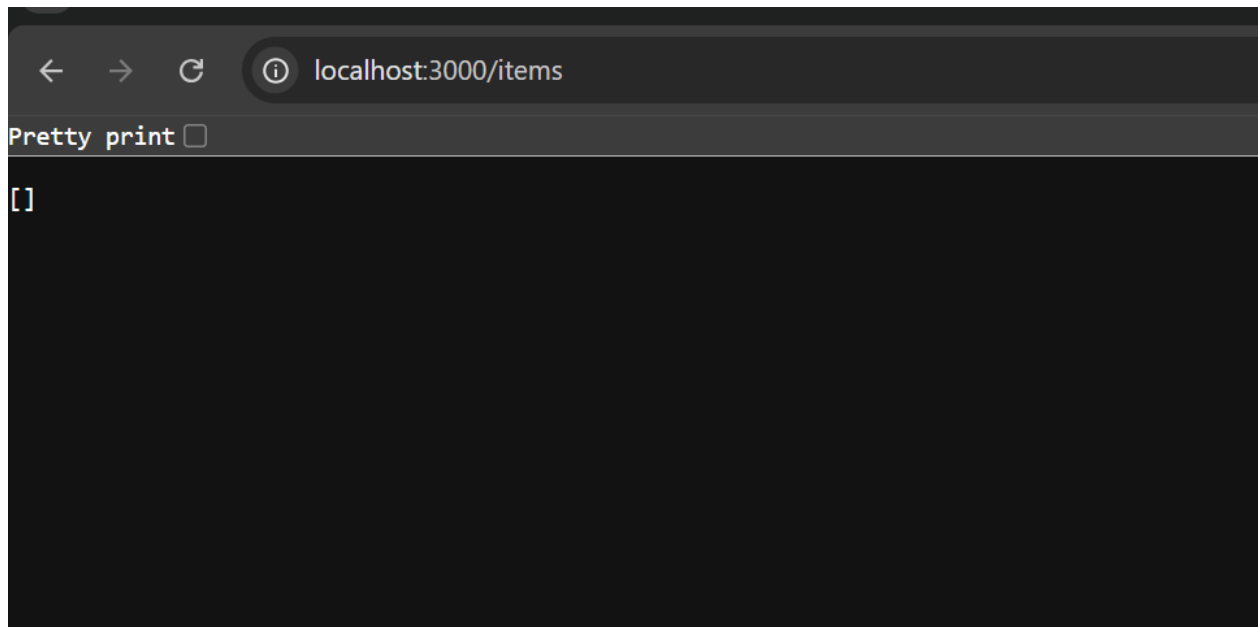
```
    removed = items.pop(index)
```

```
    return {"message": "Item removed", "item": removed}
```

Expected Output:

DELETE /items/0

→ `{"message": "Item removed", "item": {"name": "Pencil", "price": 10}}`



Task Descriptor -5: (Add Auto-Generated Documentation)

Instructions:

- Use AI to add inline comments and docstrings for all endpoints.
- Optionally, integrate Swagger / Flask-RESTX to auto-generate API documentation.

Sample Code:

```
@app.route('/items', methods=['GET'])
def get_items():
    """
```

GET /items

Returns a list of all items in the store.

```
"""
```

```
    return jsonify(items)
```

Expected Output:

- Clear documentation for all endpoints, e.g., via /docs if using Swagger.
- Students should be able to see endpoint description, methods, and sample payloads.

Items API 1.0.0 OAS 3.0

Simple CRUD API for managing items

Servers

`http://localhost:3000` ▾

default ^

GET	/items	Get all items	▾
POST	/items	Create a new item	▾
PUT	/items/{index}	Update an item	▾
DELETE	/items/{index}	Delete an item	▾



11:43 PM
3/14/26